Regular Expressions and Languages

Zhengfeng Ji, *Department of Computer Science and Technology, Tsinghua University*

1 Finite Automata With Epsilon-Transitions

It is an NFA allowed to make a transition spontaneously without receiving an input symbol.

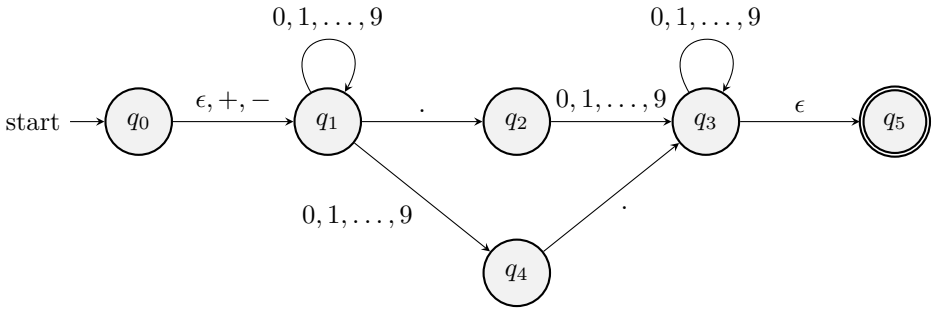
1.1 Why?

- It has the same power as DFAs and NFAs.
- But they are sometimes ***easier*** to use and construct, for example, when we discuss regular expressions.

1.2 An example

- Consider Example 2.18 of the textbook.

	ϵ	$+,-$	.	$0,1,\ldots,9$
$\to q_0$	$\{q_1\}$	$\{q_1\}$	$\emptyset$	$\emptyset$
q_1	$\emptyset$	$\emptyset$	$\{q_2\}$	$\{q_1, q_4\}$
q_2	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_3\}$
q_3	$\{q_5\}$	$\emptyset$	$\emptyset$	$\{q_3\}$
q_4	$\emptyset$	$\emptyset$	$\{q_3\}$	$\emptyset$
$*q_5$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$



An ϵ-NFA that accepts decimal numbers consisting of:

1. An optional $+$ or $-$ sign
2. A string of digits
3. A decimal point
4. Another string of digits. Either this string or the string in (2) can be empty.

- Compare it with the equivalent DFA.

1.3 Formal definition

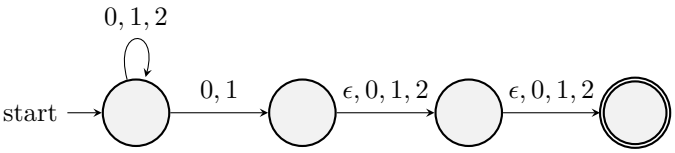
- Compared with the formal definition of an NFA, now the transition function δ takes a state and a member of $\Sigma \cup \{\epsilon\}$ as input.

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \to 2^Q.$$

1.4 Examples

Example. Design an ϵ-NFA that accepts all strings over $\{0, 1, 2\}$ whose length is at least one and in which one of the last three letters is not 2.

It means that there is a 0 or 1 in the last three letters.



Example. Design an ϵ-NFA that accepts

$L = \{w \mid w \in \{a, b, c\}^* \wedge |w| \geq 4 \wedge$
there is a bac in the first five symbols of $w \wedge$
there is an acb in the last five symbols of $w\}$.

Can you see there are ***three*** cases to consider?

1.5 Epsilon-closure ECLOSE(q)

- The key is to keep track of the possible current states of an automaton.
- What is ECLOSE(q) informally?

  Define it yourself.

  If p, q are in ECLOSE(r), do we always have ECLOSE(p) = ECLOSE(q)? No!

> The minimum set such that
>
> 1. $q \in$ ECLOSE(q).
> 2. If $p \in$ ECLOSE(q) and $r \in \delta(p, \epsilon)$, then $r \in$ ECLOSE(q).

- ECLOSE(S) is the union of ECLOSE(q) for $q \in S$.
- Example
  1. ECLOSE(q_0) = $\{q_0, q_1\}$
  2. ECLOSE(q_2) = $\{q_2\}$

1.6 Extended transition

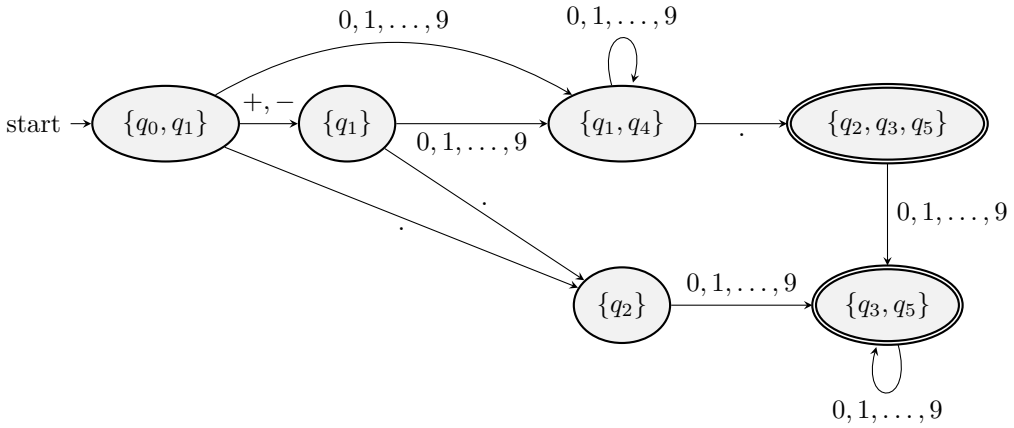
- Take an ϵ move at the end.
- Basis: $w = \epsilon$, it is ECLOSE(q).
- Induction: $w = xa$
  1. Take the x transition,
  2. Take the union with a,
  3. Take the ϵ-closure.

1.7 Eliminate ϵ-transitions

- For any ϵ-NFA A, there is a DFA B equivalent to A. (Not that different from the equivalence of DFA and NFA)
  1. Compute the ECLOSE.
  2. Use the ***subset construction*** (with ECLOSE after each move)

  Often, you do not need all the subsets! Start from the initial state and generate the subsets one by one.
- Example

  The DFA obtained by eliminating the ϵ-transitions in Example 2.18 is given in the figure above. For simplicity, the dead state and arrows to it are not shown.



> The starting state is ECLOSE(q_0) = $\{q_0, q_1\}$.

`grep`, `lex`

2 Regular Expressions

> Now we switch our attention from machine-like descriptions of languages—deterministic and nondeterministic finite automata—to an ***algebraic*** description of the regular expression.
>
> And show that they can describe all and only regular languages (languages accepted by finite automata).

It is a ***declarative*** way to express the strings we want to accept.

2.1 Regular expressions: Definition

- Basis: three parts
  1. The constants ϵ and $\emptyset$ are regular expressions. $L(\epsilon) = \{\epsilon\}$.
  2. If a is a symbol, then a is a RE. $L(a) = \{a\}$.
- Induction: four parts
  1. If E and F are REs, then so is the union $E + F$. $L(E + F) = L(E) \cup L(F)$.
  2. ..., then so is the concatenation EF. $L(EF) = L(E)L(F)$.
  3. ..., then so is the closure E^*. $L(E^*) =$

$(L(E))^*$.

4. …, then so is (E). $L((E)) = L(E)$.

2.2 Examples

01 means $\{01\}$

01^* means $\{0, 01, 011, 0111, \ldots\}$

(01)^* means $\{\epsilon, 01, 0101, \ldots\}$

Example. Write a RE for strings with alternating 0's and 1's.

$(01)^* + (10)^* + 1(01)^* + 0(10)^*$

or

$(\epsilon + 1)(01)^*(\epsilon + 0)$

Example. Write a RE for binary strings whose first and last letters are the same.

$0(0 + 1)^*0 + 1(0 + 1)^*1 + 0 + 1$

Example. Write a RE for binary strings where every pair of neighboring 0's occurs before every pair of neighboring 1's.

Strings in the language can be partitioned into two parts where the left part has no 11, and the right part has no 00.

$(10 + 0)^*(\epsilon + 1)(01 + 1)^*(\epsilon + 0)$

2.3 Precedence of regular expression operators

$xy + z$

Order: star, dot, union

3 Application of RE

Read Section 3.3 of the textbook.

3.1 Unix extension

1. The ability to name and refer to previous strings that have matched a pattern.
2. More about a way of writing RE succinctly.
   (a) dot . stands for any character
   (b) $[a_1 a_2 \cdots a_k]$ stands for $a_1 + a_2 + \cdots + a_k$
   (c) range form [A-Za-z0-9]
   (d) [:digit:], …
3. Notation
   (a) | is used instead of +
   (b) ? zero or one ($R? = \epsilon + R$)
   (c) + one or more ($R^+ = RR^*$)
   (d) $\{n\}$ n copies of

3.2 Lexical analysis

```
else                    {return (ELSE);}
[A-Za-z][A-Za-z0-9]*    {code to enter the found identifier to the symbol table; return (ID);}
>=                      {return (GE);}
```

3.3 Finding patterns in text

Most editors support RE search and replacement.

4 Finite Automata and Regular Expressions

Define exactly the same set of languages.

Perhaps the most important result in FA & RE.

4.1 DFA to RE

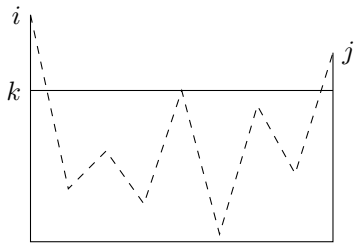
- ***Kleene construction***

  Induction + Restrict the size + Consider expressions between multiple states

  Probably the first ***non-trivial*** proof we see in this course.

  Consider several languages together.

- Idea: Roughly, we build expressions that describe sets of strings that label certain paths in the DFA's transition diagram. However, the paths can pass through only a limited subset of the states. In an inductive definition of these expressions, we start with the simplest expressions that describe paths that are not allowed to pass through any states. And inductively build the expressions that let the paths go through progressively larger sets of states.



- Proof outline

  Proof. Let the states be $\{1, 2, \ldots, n\}$.

  Let $R_{ij}^{(k)}$ be the strings such that if we start from i, we will end up at j visiting ***intermediate*** (not including i, j) states $\leq k$.

  Basis: $k = 0$. Since all states are numbered 1 and above, the restriction on the path is that the path must have no intermediate states at all.

  There are ***two*** cases to consider.

  1. An arc from i to j.
  2. A path of length 0 that consists of state i only.

  If $i \neq j$, then only case 1 is possible.

  In that case, $R_{i,j}^{(0)}$ is $a_1 + a_2 + \cdots + a_k$ where there are transitions from i to j on symbols $a_1, a_2, \ldots, a_k$.

  If $i = j$, consider all the loops from i to itself.

  $$R_{ij}^{(0)} = \begin{cases} \epsilon & \text{no self loop,} \\ \epsilon + a_1 + \cdots + a_k & \text{loops with } a_1, \ldots, a_k \end{cases}$$

  Induction: (on k)

  1. The path does not go through state k at all.
  2. The path goes through state k at least once.

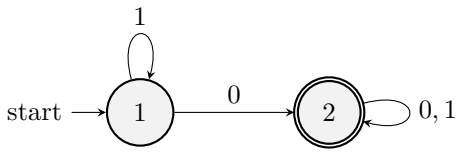
  $$R_{ij}^{(k)} = R_{ij}^{(k-1)} + R_{ik}^{(k-1)} \left(R_{kk}^{(k-1)}\right)^* R_{kj}^{(k-1)}.$$

  □

- The method works even for NFAs.

4.2 Example

- Convert the DFA in the figure to a RE.



Some intermediate expressions are shown in the table.

	Regular Expression
$R_{22}^{(0)}$	$\epsilon + 0 + 1$
$R_{12}^{(1)}$	1^*0
$R_{22}^{(1)}$	$\epsilon + 0 + 1$
$R_{12}^{(2)}$	$1^*0(0+1)^*$